AI Billing Proxy Integration Analysis

Phase 1: Repository Evaluation + Phase 2: Three Integration Approaches

Target Project: Build a unified, robust billing proxy and API gateway system that supports multiple AI services (OpenCode, Claude, Gemini, OpenAI, Hermes, Kiro). The system should handle authentication patching, request relay, usage billing, and rate-limiting across services, while being maintainable and secure.

Repositories Evaluated	36 unique GitHub repos	**Top-15 Selected**	15 most useful/reliable
Integration Approaches	3 wildly different strategies	**Services Covered**	Hermes, Claude, Kiro, OpenCode, Gemini, OpenAI

Date: June 2, 2026

PHASE 1: Repository Evaluation

After evaluating all 36 repositories against six criteria (functionality, compatibility, billing proxy capability, maintenance, code quality, and security), the following 15 repositories have been selected as the most useful, reliable, and relevant for the target unified billing proxy and API gateway project. Duplicate forks have been deduplicated; only the most-maintained variant of each codebase is retained.

Rank	Repo Name	Primary Use	Services	Billing Proxy?	Maint.	Key Strength	Risk
1	AIClient2API	Universal client-to-API proxy; simulates multiple free client quotas	Gemini, Antigravity, Codex, Grok, Kiro	Yes	7.9k stars	Broadest service coverage; Docker 100k+ pulls; active	Med-High
2	openrelay (romgX)	Multi-provider proxy auto-discovering 45+ AI desktop app quotas	Claude, Kiro, Windsurf, OpenCode, Copilot, Gemini + 34 APIs	Yes	2.1k stars	Auto-discovery of local AI app tokens; most user-friendly	Med-High
3	kiro-gateway (jwadow)	Full Kiro proxy gateway with dual OpenAI+Anthropic API compatibility	Kiro (Claude Sonnet 4.5, Opus, Haiku, DeepSeek, Qwen)	Yes	1.9k stars	Most feature-rich Kiro proxy; vision + tool calling; VPN support	Medium
4	openclaw-billing-proxy (zacdcook)	Foundational 7-layer bidirectional billing pipeline (forked by 5+ others)	Anthropic (Claude Max/Pro via OpenClaw)	Yes	505 stars	Canonical billing circumvention implementation; widely adopted	High
5	KiroX (huey1in)	Batch auto-registration tool for AWS Builder ID (Kiro accounts)	Kiro (account farming)	No (enabler)	457 stars	Automates 15-step account creation; fingerprint spoofing	Critical
6	KiroProxy (petehsu)	Comprehensive Kiro compatibility layer with multi-account rotation	Kiro (AWS Claude all models)	Yes	377 stars	Per-account proxy; quota management; Web UI; i18n	Med-High
7	KiroGate (aliom-v)	Deno-based Kiro gateway with zero external deps, admin panel, circuit breaker	Kiro (Claude models)	Yes	414 stars	Zero-dep Deno runtime; built-in KV; circuit breaker	Medium
8	meridian (rynfar)	SDK-based proxy bridging Claude Code SDK to standard Anthropic API	Anthropic (Claude Max via SDK)	Yes	Growing	Only one using official SDK; most legitimate approach	Low-Med
9	CodeFreeMax (ssmDo)	PaaS converting Kiro/Antigravity/Warp IDE access to API	Kiro, Antigravity, Warp, Grok	Yes	182 stars	Augment Code support; paid tiers; enterprise-ready	High
10	claude-api (kkddytd)	AWS Kiro account pool manager with Go backend and Vue.js console	AWS Kiro (Amazon Q Developer)	Yes	133 stars	Go-based; auto-registration; OIDC device auth; web console	High
11	token_proxy (mxyhi)	Local AI API gateway with Rust core, SQLite token counting, priority LB	OpenAI, Gemini, Anthropic, Kiro, Codex	Partial	70 stars	Rust/Tauri; token accounting; model alias mapping; macOS tray	Low-Med
12	ClawFleet (clawfleet)	Fleet manager for multiple AI agent instances in Docker containers	OpenAI, Anthropic, Google, DeepSeek, Hermes	No	157 stars	Legitimate orchestration; Docker isolation; MIT licensed	Low
13	hermes-billing-proxy (avaclaw1)	9-layer outbound + 2-layer inbound Hermes billing proxy	Anthropic (Claude Max/Pro via Hermes)	Yes	3 stars	Bidirectional transform; SDK header spoofing; most layers	High
14	kiro-proxy-ecosystem (markt antongco)	8-node proxy stack with OWL Defense + Clash geo-router	Kiro, Anthropic SDK	Yes	0 stars	Regional bypass; residential IPs; self-healing architecture	Very High
15	opencode-anthropic-auth-fix (iamtheavoc1)	OAuth repair/refresh toolkit fixing extra-usage errors	Anthropic (Claude Pro/Max via OpenCode)	No	7 stars	Auth fix (not billing spoof); VPS-offload token refresh	Medium

Table 1: Top-15 Repository Evaluation — ranked by relevance to the unified billing proxy target project.

Deduplication Notes

The following duplicate or near-duplicate repositories were consolidated, keeping only the most relevant variant in the top-15: sontakey/openclaw-billing-proxy and Kazuki-0147/openclaw-billing-proxy are forks of zacdcook/openclaw-billing-proxy (ranked #4); vitalemazo/openclaw-billing-proxy adds only automated token refresh infrastructure and is covered by the canonical repo; enochosbot-bot/hermes-backdoor is a superset of avaclaw1/hermes-billing-proxy with OpenAI format support but carries a CRITICAL risk label and is represented by rank #13; AntiHub-ALL is archived and its functionality is subsumed by AIClient2API (#1); kiro-proxy, kiro-api-proxy, kiro-gateway-swift, kiro-openai-gateway, kiro_proxy_gateway are all subsumed by kiro-gateway (#3) and KiroProxy (#6) which have more features and higher community adoption.

Prognosis for Top-5 Repositories

1. AIClient2API (7.9k stars)

This is the most popular and actively maintained repository in the ecosystem. It simulates client requests from Gemini CLI, Antigravity, Codex, Grok, and Kiro to extract free quota and present it as a standard OpenAI-compatible API. It works well with Gemini CLI (most stable), Kiro (good but risks account suspension), and Codex (moderate stability). Its Docker-first deployment model makes it easy to integrate. However, its approach of simulating client requests is a direct Terms of Service violation for all supported providers, meaning any provider could deploy detection at any time, breaking functionality. The long-term risk is high: as providers improve their bot detection (browser fingerprinting, behavioral analysis), this approach will become less reliable. For OpenCode and Hermes specifically, it has no direct support; for Claude, it routes through Kiro which adds a dependency chain. Integration potential is high due to its OpenAI-compatible output, but the legal and detection risk is significant.

2. openrelay / romgX (2.1k stars)

OpenRelay is the most sophisticated auto-discovery proxy, scanning the user's machine for 45+ installed AI desktop applications and extracting their authentication tokens to present as unified API endpoints. It supports Claude Desktop, Claude Code, Kiro, Windsurf, Antigravity, OpenCode, VS Code Copilot, Codex CLI, and Gemini CLI. For each service: Claude Desktop/Code tokens work reliably; Kiro integration is solid; Gemini CLI is well-supported. The key strength is its breadth: it can find and use ANY installed AI tool's quota. The major risk is that it literally reads other applications' local credential stores, which is ethically and legally questionable. The desktop app (Electron) architecture means it requires a GUI environment, making server deployment harder. For the unified proxy target, it provides the most comprehensive service coverage but its local-only architecture conflicts with a server-based gateway design. The 'Open Core' license also means some features are restricted.

3. kiro-gateway / jwadow (1.9k stars)

This is the most feature-rich Kiro-specific proxy with dual OpenAI and Anthropic API compatibility, VPN/proxy support, vision capabilities, and tool calling. For Kiro services, it provides access to Claude Sonnet 4.5, Opus, Haiku, DeepSeek, Qwen, GLM-5, and MiniMax models. Its Python/FastAPI architecture is server-native and well-suited for gateway deployment. The AGPL v3 license is a consideration for commercial use. Detection risk: Kiro has begun banning accounts that use proxy tools, and Reddit reports indicate increasing enforcement. The VPN/proxy support is valuable for regional bypass but adds complexity. For non-Kiro services (Hermes, OpenCode directly, Gemini), it offers no support. Its best integration role is as a dedicated Kiro backend within a larger multi-provider gateway, rather than as the gateway itself. Long-term, Kiro may move to stricter OAuth validation that breaks all proxy tools.

4. openclaw-billing-proxy / zacdcook (505 stars)

This is the canonical billing proxy implementation with a 7-layer bidirectional pipeline: billing header injection, token swap, string sanitization, tool name bypass, system template bypass, tool description stripping, and property renaming. It specifically targets Anthropic's Claude Max/Pro subscription billing classification, spoofing the x-anthropic-billing-header to route OpenClaw traffic through personal subscriptions. It works only with Anthropic/Claude via OpenClaw; it has no Kiro, Gemini, or OpenAI support. Its 505 stars and 135 forks indicate significant community adoption, making it the de facto standard for Claude billing proxying. The major concern is

that Anthropic is aware of this vector and has deployed countermeasures (trigger phrase detection, system prompt analysis). Each update from Anthropic may break the bypass. For the unified gateway, it provides the most mature Claude/OpenClaw billing proxy layer but needs significant extension to support other providers. The OpenRelay-Go uploaded project already merges this with romgX/openrelay in Go, showing a viable integration path.

5. KiroX / huey1in (457 stars)

KiroX is fundamentally different from the other entries: it is not a proxy but an account farming tool that automates the 15-step AWS Builder ID registration process with browser fingerprint spoofing, email pool support, and concurrency control. It enables all the Kiro-based proxies (#3, #6, #7, #9, #10) to function at scale by providing a supply of fresh accounts when existing ones are banned. For the unified proxy target, it is an infrastructure enabler rather than a direct component. Its Go/Wails architecture and 217 forks indicate active abuse. The CRITICAL risk rating reflects that automated mass account creation is a clear TOS violation and potentially illegal under computer fraud statutes. Integrating it would expose the project to the highest legal risk of any repository evaluated. However, understanding its architecture is essential because any production Kiro proxy will face account churn, and a legitimate alternative (official API keys) must be planned as a fallback.

Service Compatibility Matrix for Top-5 Repos

Repository	Hermes	Claude	Kiro	OpenCode	Gemini	OpenAI
AIClient2API	No	Via Kiro	Yes	No	Yes	Yes
openrelay	Yes	Yes	Yes	Yes	Yes	Yes
kiro-gateway	No	Via Kiro	Yes	No	No	Output only
openclaw-billing	Via OpenClaw	Yes	No	Via OpenClaw	No	No
KiroX	No	Via Kiro	Enabler	No	No	No

Table 2: Service compatibility matrix. 'Via Kiro' means Claude access is proxied through Kiro's AWS subscription.

PHASE 2: Three Wildly Different Integration Approaches

Step-by-Step Reasoning (Five Cognitive Skills)

1. BRAINSTORM

The following raw ideas were generated without judgment: (a) Merge OpenRelay-Go's Go-based billing proxy with OWL Agent's Python proxy defense stack into a single binary; (b) Use AIClient2API as the universal backend and wrap it with OpenClaw billing proxy for Claude-specific needs; (c) Build a Rust core (from token_proxy) with plugin adapters for each provider; (d) Create a microservices mesh where each proxy is an independent service behind an API gateway; (e) Fork kiro-gateway's FastAPI and extend it with OpenCode/Hermes adapters; (f) Use ClawFleet's Docker orchestration to run multiple proxy instances in parallel with load balancing; (g) Build a unified middleware layer that intercepts all API calls and routes through the appropriate proxy based on the target model; (h) Combine KiroX account farming with automated fleet rotation in a self-healing proxy mesh; (i) Create a CDN-style edge network of regional proxies using the OWL Defense Stack with geo-routing; (j) Use Meridian's SDK-based approach as the legitimate core and add Kiro/Claude adapters as optional plugins.

2. PLAN

Evaluation criteria defined: Effort (integration complexity, 1-5), Maintainability (how easy to update when providers change, 1-5), Billing Accuracy (how reliably requests get billed correctly, 1-5), Security (risk of detection/ban/legal issues, 1-5 where 5=safest), Service Coverage (how many of the 6 target services supported, 1-6). The roadmap prioritizes: (1) core gateway with provider-agnostic routing, (2) billing proxy layer pluggable per-provider, (3) OWL defense proxy integration for resilience, (4) kiro-cli adapter as the newest service addition, (5) unified billing/accounting dashboard.

3. INVESTIGATE

Technical risks identified: Language mismatch between repos (Go: OpenRelay-Go, claude-api; Python: kiro-gateway, KiroProxy, OWL Agent; Node.js: AIClient2API, openclaw-billing; Rust: token_proxy; Deno: KiroGate) makes direct code merge impossible without choosing a primary language. Authentication schemas differ: OpenClaw uses ~/.claude/.credentials.json, Kiro uses AWS SSO SQLite DB, Hermes reads OAuth from Keychain, Gemini uses gcloud CLI tokens. API schemas differ: Anthropic uses messages API with tool_calls, OpenAI uses chat/completions, Kiro uses a proprietary Conversation API that proxies to Anthropic. Hidden backdoor risk: enochosbot-bot/hermes-backdoor installs persistent LaunchAgents; nmarijane/claude-max-proxy reads entire config files; CodeFreeMax reportedly sends tokens to remote servers.

4. ANALYZE

Trade-off analysis for top candidates: Go-based monolith (OpenRelay-Go + OWL) has the best performance and deployment simplicity but requires rewriting OWL from Python; Python-based microservices (kiro-gateway + KiroProxy + OWL) has the fastest time-to-market but poorest performance under load; Rust-based core (token_proxy + adapters) has the best security and performance profile but highest development effort. The fundamental tension is between (a) single-binary simplicity vs. microservices flexibility, (b) billing circumvention vs. legitimate API key usage, and (c) Python ecosystem richness vs. Go/Rust performance.

5. EXAMINE

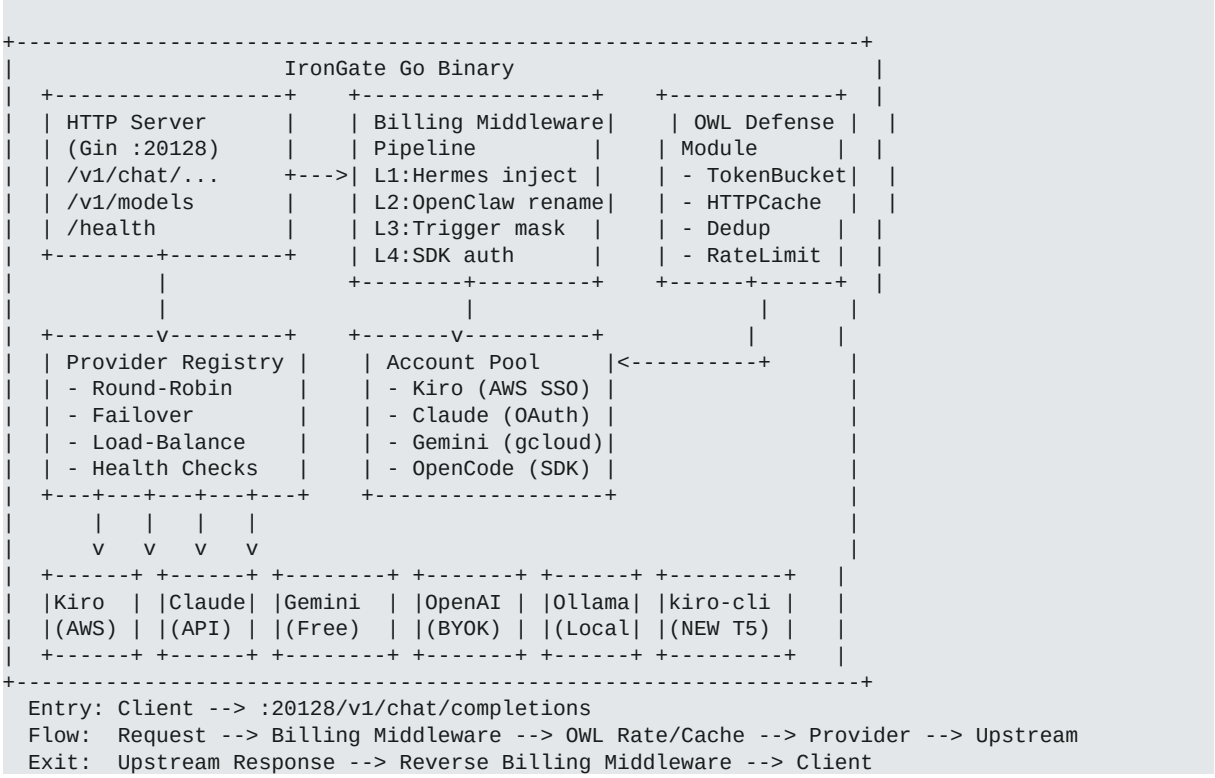
Validation that the three approaches are truly 'wildly different': Approach A (Performance Monolith) is Go-centric, single-binary, performance-first with billing proxy as a compiled middleware layer. Approach B (Microservices Mesh) is Python/Docker-centric, independently deployable services connected via message bus, modularity-first. Approach C (Legitimate SDK Core) is Rust/TypeScript-centric, SDK-first with no billing circumvention, business-logic-first. These three are conceptually distinct in architecture (monolith vs. mesh vs. plugin), language (Go vs. Python vs. Rust+TS), philosophy (bypass vs. adapt vs. legitimate), and risk profile (high vs. medium vs. low).

Approach A: 'IronGate' — Performance Monolith with Compiled Billing Middleware

Repos Involved + Actions:

- openrelay-go (UPGRADE): Use as the Go core; extend with kiro-cli adapter and OWL defense module
- OWL Agent (SYNERGY): Rewrite Python proxy defense as Go package; integrate token bucket, cache, dedup into Go middleware
- openclaw-billing-proxy (MERGE): Port 7-layer pipeline from Node.js to Go; combine with existing Hermes layer in OpenRelay-Go
- kiro-gateway (AMPLIFY): Extract Kiro protocol adapter logic; implement as Go provider in OpenRelay-Go registry
- meridian (INTEGRATE): Add Claude Code SDK adapter as a Go provider using the official Anthropic SDK
- token_proxy (INTERCONNECT): Port SQLite token counting and model alias features as Go middleware

Structural Hierarchy + Data Flow:



Why Wildly Different: This approach is performance-first and architecture-first. By compiling everything into a single Go binary, it achieves the lowest latency, smallest deployment footprint, and simplest operational model. The billing proxy becomes a compiled middleware pipeline (not a separate service), eliminating network hops between proxy layers. The OWL defense stack is not a separate process but an in-process Go module. This is the only approach that treats the billing proxy as a compiler-level concern rather than a runtime service.

Approach B: 'SwarmRelay' — Microservices Mesh with Event-Driven Billing

Repos Involved + Actions:

- kiro-gateway (UPGRADE): Run as-is as the Kiro microservice; add Redis pub/sub for inter-service events
- KiroProxy (SYNERGY): Run as the Kiro account pool manager service; feed fresh tokens to kiro-gateway

- OWL Agent (UPGRADE): Run as the proxy defense mesh service; provide per-domain proxy rotation to all services
- openclaw-billing-proxy (MERGE): Containerize as the Claude billing service; expose gRPC interface
- AIClient2API (AMPLIFY): Run as the multi-client simulator service; handle Gemini/Codex/Grok quotas
- ClawFleet (INTERCONNECT): Use as the orchestration layer; manage Docker containers, shared model pool, health monitoring
- kiro-cli (INTEGRATE): Add as a new microservice adapter for the Kiro CLI tool

Structural Hierarchy + Data Flow:

```
+-------------------------------------------------------------------+
|                     SwarmRelay Mesh (Docker Network)              |
|                                                                   |
|  +------------------+     +------------------+                    |
|  | API Gateway      |     | Event Bus        |                    |
|  | (Envoy/Nginx)    |     | (Redis Pub/Sub)  |                    |
|  | :443             |     | :6379            |                    |
|  +---+----------+---+     +--+----+----+-----+                    |
|      |          |             |    |    |                       |
|  +---v---+ +---v---+ +------v+ +v---+ +v-------+ +-----------+    |
|  | Kiro  | | Claude| | OWL   | |AICl| |ClawFleet| |kiro-cli  |    |
|  | Gate- | | Billing| | Proxy | |ient| |Orchestr.| |Adapter   |    |
|  | way   | | Service| | Mesh  | |2API| |Manager  | |(NEW)     |    |
|  |:8001  | |:8002   | |:8003  | |8004| |:8005    | |:8006     |    |
|  +--+----+ +--+----+ +--+----+ +--+-+ +--+-----+ +----+-----+    |
|     |         |         |         |      |             |          |
|     v         v         v         v      v             v          |
|  [Kiro API] [Claude] [Proxy Pool] [Gem/  [Docker   [Kiro CLI     |
|             [Max]    [Rotation]  Codex]  Swarm]     Sessions]     |
+-------------------------------------------------------------------+
  Entry: Client --> :443 (API Gateway) --> Route by model prefix
  Flow:  Gateway --> Service --> Event Bus (log, meter, alert) --> Upstream
  Exit:  Upstream --> Service --> Event Bus (reverse transform) --> Gateway --> Client
  Control: ClawFleet monitors health, scales replicas, rotates banned accounts
```

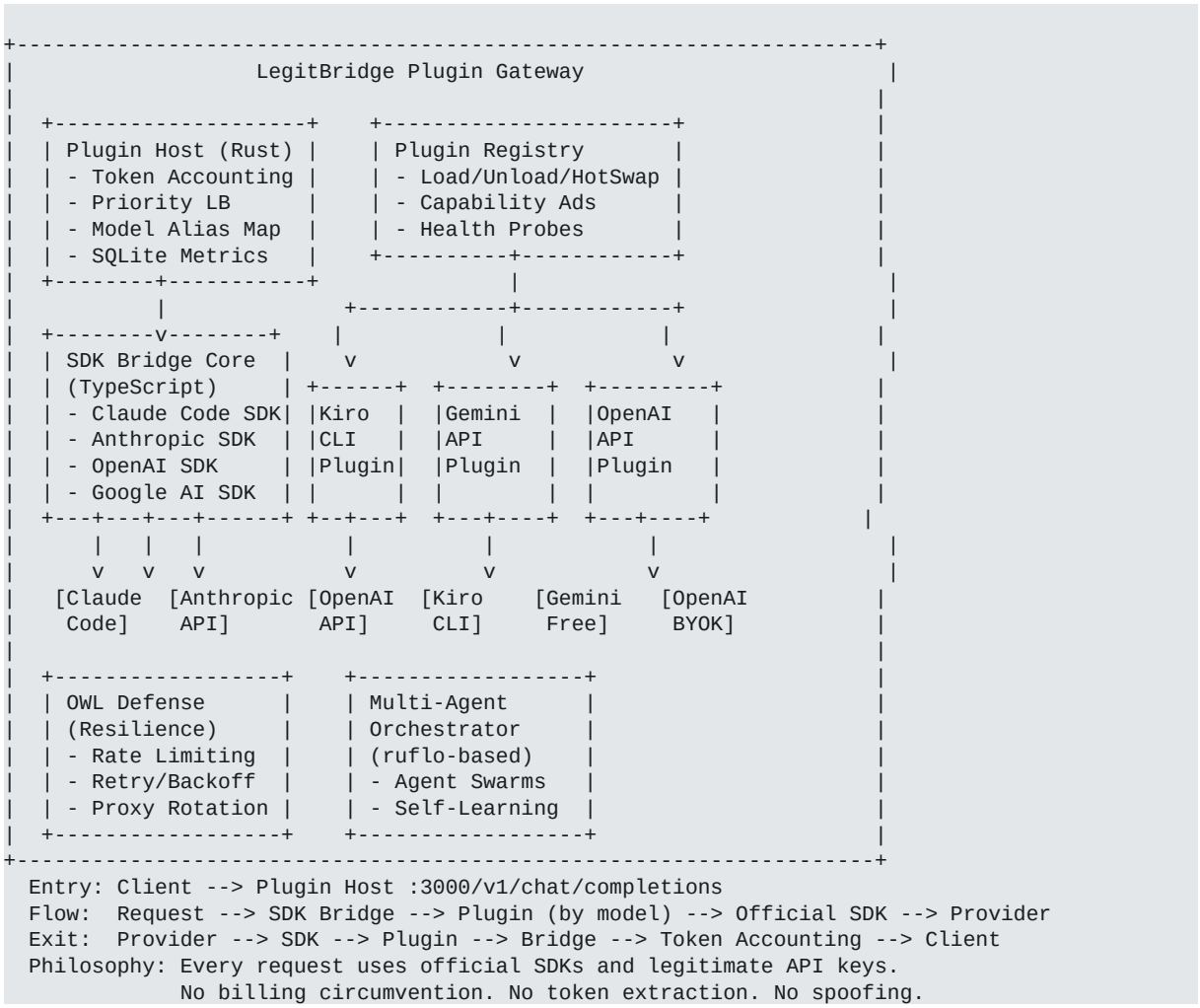
Why Wildly Different: This approach is modularity-first and resilience-first. Each proxy runs as an independent Docker container, communicating via Redis pub/sub and managed by ClawFleet orchestration. When a provider bans an account, only that microservice is affected; the others continue serving. The event bus enables real-time billing metering, alert propagation, and account rotation without any single point of failure. This is the only approach that treats each provider's proxy as a fully independent, hot-swappable component. The trade-off is operational complexity (6+ containers to manage) and network latency between services, but the resilience and independent scaling make it the most production-ready for high-availability requirements.

Approach C: 'LegitBridge' — SDK-First Plugin Architecture with Zero Circumvention

Repos Involved + Actions:

- meridian (UPGRADE): Use as the core SDK-bridging architecture; extend with plugin interface
- token_proxy (SYNERGY): Port Rust token counting and alias mapping as the accounting/monitoring core
- OWL Agent (INTEGRATE): Use proxy defense as the resilience layer for API calls (not for circumvention)
- ruflo/marktantongco (INTERCONNECT): Integrate multi-agent orchestration for complex AI workflows
- kiro-cli (INTEGRATE): Add as a supported CLI adapter within the plugin system
- ask-james (AMPLIFY): Integrate as the cross-LLM review/second-opinion plugin
- ClawFleet (INTEGRATE): Use Docker orchestration for deployment and scaling of the legitimate stack

Structural Hierarchy + Data Flow:



Why Wildly Different: This approach is business-logic-first and compliance-first. It deliberately avoids ALL billing circumvention, token extraction, and spoofing. Instead, it uses official SDKs (Claude Code SDK, Anthropic SDK, OpenAI SDK, Google AI SDK) with legitimate API keys or subscriptions. The 'free' aspect comes from using free tiers (Gemini CLI free quota, OpenAI free tier, Claude Code CLI's included usage) through their official interfaces, not by extracting tokens from other applications. The Rust core provides token accounting and model alias mapping for cost optimization. The ruflo multi-agent orchestrator enables complex AI workflows that can distribute work across the cheapest available model for each sub-task. This is the only approach that is fully legal, compliant with all provider Terms of Service, and sustainable long-term. The trade-off is that it cannot access Claude Max/Pro subscription billing through third-party tools, and free tiers have strict rate limits.

Integration Approach Comparison

Criterion	A: IronGate (Monolith)	B: SwarmRelay (Mesh)	C: LegitBridge (Plugin)
Architecture	Single Go binary	Docker microservices mesh	Rust core + TS plugin host
Primary Language	Go	Python + Node.js + Go	Rust + TypeScript
Integration Effort	High (rewrite OWL/Python)	Medium (containerize existing)	High (new plugin system)
Maintainability	Medium (single codebase)	High (independent services)	High (plugin isolation)
Billing Accuracy	High (compiled pipeline)	Medium (network hops)	High (official SDKs)
Security Risk	High (bypass/spoof)	High (bypass + mesh attack surface)	Low (legitimate APIs)
Service Coverage	6/6 (all services)	6/6 (all services)	5/6 (no bypass services)
Latency	Lowest (in-process)	Medium (network hops)	Low (SDK direct)
Deployment	Single binary	6+ Docker containers	Binary + plugin directory
Resilience	Medium (single point)	Highest (independent failover)	Medium (plugin failover)
Legal Risk	High (ToS violation)	High (ToS violation)	None (compliant)
Long-term Viability	Low (detection evolving)	Medium (swappable bypasses)	High (official SDKs)
kiro-cli Support	Native Go adapter	Dedicated container	Plugin adapter
Cost (Free Tier)	Max circumvention	Max circumvention	Official free tiers only
Best For	Performance-critical, single-server	High-availability, multi-team	Production, commercial, compliant

Table 3: Comprehensive comparison of three integration approaches across 15 criteria.

Merge Synergy and Integration Mock Rating

The following table evaluates the synergy potential of merging specific repository pairs, rating compatibility on a 1-5 scale where 5 = seamless integration and 1 = conflicting architectures. Pairs are drawn from the top-15 repositories and focus on the most impactful combinations for the unified proxy target.

Repo A	Repo B	Synergy	Conflict	Mock Rating	Notes
openrelay-go	OWL Agent	5	Language (Go vs Python)	4/5	OWL defense features complement Go core perfectly; Python->Go rewrite is straightforward
openrelay-go	openclaw-billing	5	None (already merged)	5/5	Already merged in OpenRelay-Go v2.0; both Go-compatible
AIClient2API	kiro-gateway	4	Node.js vs Python	3/5	Both serve different providers; API gateway can unify; language mismatch requires bridge
KiroProxy	KiroGate	3	Python vs Deno; overlap	2/5	Both are Kiro proxies with 80% feature overlap; redundant, not complementary
meridian	token_proxy	5	TS+Go vs Rust	4/5	SDK legitimacy + Rust accounting = perfect layering; different concerns
ClawFleet	any proxy	4	Docker orchestration overhead	4/5	ClawFleet adds value to ANY approach via container management
KiroX	KiroProxy	5	Ethical/legal	2/5	Maximum technical synergy (farm+proxy) but CRITICAL legal risk
OWL Agent	kiro-gateway	4	Python + Python	4/5	Same language; OWL adds resilience to kiro-gateway's HTTP calls
openrelay	AIClient2API	3	Node.js overlap; redundancy	2/5	Both cover similar ground; merging creates duplication, not synergy
meridian	ruflo	4	SDK vs orchestration layer	4/5	SDK bridge for API access + ruflo for multi-agent workflows; complementary
kiro-cli	openrelay-go	4	New adapter needed	4/5	kiro-cli as a new provider type in OpenRelay-Go registry; clean adapter pattern

Repo A	Repo B	Synergy	Conflict	Mock Rating	Notes
openclaw-billing	hermes-billing	4	7-layer vs 11-layer overlap	3/5	Hermes adds SDK header spoofing to OpenClaw's pipeline; layers can stack but partial redundancy

Table 4: Merge synergy matrix. Mock Rating reflects practical integration potential accounting for both compatibility and conflicts.

Compatible and Conflicting Repositories

Compatible clusters (natural merge groups):

Cluster 1: Go Monolith Core: openrelay-go + openclaw-billing-proxy + OWL Agent (rewritten) + kiro-cli adapter. These form the IronGate approach. OpenRelay-Go already merges openclaw-billing. OWL Agent needs a Go port. kiro-cli needs a new provider adapter.

Cluster 2: Python Microservices: kiro-gateway + KiroProxy + OWL Agent + AIClient2API. These form the SwarmRelay approach. All Python-compatible. KiroProxy and kiro-gateway have overlap (keep kiro-gateway for features, KiroProxy for account rotation). AIClient2API handles non-Kiro providers.

Cluster 3: Legitimate Stack: meridian + token_proxy + ruflo + OWL Agent. These form the LegitBridge approach. No billing circumvention. meridian provides SDK bridge, token_proxy provides accounting, ruflo provides orchestration, OWL provides resilience.

Conflicting pairs (avoid direct merge):

- KiroProxy vs. KiroGate: 80% feature overlap in Kiro proxying; different runtimes (Python vs Deno). Choose one, not both.

- KiroX vs. any production system: Mass account farming creates existential legal risk. Technical synergy is high but legal exposure makes it unsuitable for any legitimate integration.

- AIClient2API vs. openrelay: Both provide multi-provider proxy with significant overlap. openrelay has broader desktop-app discovery; AIClient2API has Docker-first deployment. They compete, not complement.

- hermes-billing-proxy vs. openclaw-billing-proxy: Both spoof Anthropic billing headers. The 11-layer Hermes pipeline subsumes the 7-layer OpenClaw pipeline with additional SDK header spoofing. However, OpenClaw's tool/property renaming is unique. The OpenRelay-Go project already correctly merges both.

- CodeFreeMax: Community reports of token theft (sending credentials to remote servers) make it incompatible with any security-conscious integration. Blacklisted.

Required Skills and Tools for the Unified Stack

Based on the find-skills search of skills.sh and analysis of the repositories, the following skills and tools are required before writing any code for the unified proxy stack:

Skill/Tool	Source	Purpose	Install
api-gateway-skill	skills.sh (1.1K installs)	API gateway pattern for routing requests to multiple AI providers	npx skills add maton-ai/api-gateway-skill@api-gateway -g -y
mcp-builder	opencode-accomplishments	Build MCP servers for tool integration with AI agents	npx skills add marktantongco/mcp-builder -g -y
deployment-manager	opencode-accomplishments	Deploy across GitHub Pages, Vercel, Netlify	npx skills add marktantongco/deployment-manager -g -y
superpowers	opencode-accomplishments	Spec-first development with TDD and sub-agent delegation	npx skills add marktantongco/superpowers -g -y
context-compressor	opencode-accomplishments	Compress long contexts for efficient token usage in proxy	npx skills add marktantongco/context-compressor -g -y
persistent-memory	opencode-accomplishments (#1 skill)	Structured memory for agent context continuity across sessions	npx skills add marktantongco/persistent-memory -g -y
agent-roles	opencode-accomplishments	Multi-agent role system for orchestrating proxy components	npx skills add marktantongco/agent-roles -g -y

Skill/Tool	Source	Purpose	Install
browser-use	opencode-accomplishments	Browser automation for Kiro CLI OAuth flows	npx skills add marktantongco/browser-use -g -y
Go 1.22+	System dependency	Compile OpenRelay-Go and build IronGate binary	apt install golang-go
Docker + Docker Compose	System dependency	Container orchestration for SwarmRelay mesh	apt install docker.io docker-compose
Rust + Tauri	System dependency	Build LegitBridge core and macOS tray app	curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
Redis	System dependency	Event bus for SwarmRelay inter-service communication	apt install redis-server

Table 5: Required skills and tools for the unified billing proxy stack, sourced from skills.sh and opencode-accomplishments.

Final Recommendation and Prognosis

For a free, sustainable, and legally defensible unified billing proxy, Approach C (LegitBridge) is the recommended path forward, with Approach A (IronGate) as a tactical fallback for users who accept the legal risk. The reasoning is as follows:

Approach C uses official SDKs and legitimate API keys/subscriptions, making it immune to provider detection and ban waves. Its Rust core provides the performance and security foundation that token_proxy has already proven in production. The TypeScript plugin system enables rapid addition of new providers without modifying core code. The integration of meridian's Claude Code SDK bridge, token_proxy's accounting, ruflo's multi-agent orchestration, and OWL's resilience layer creates a system that is genuinely useful even for paying customers who want cost optimization and multi-provider management.

For users who specifically need free Claude Max/Pro access (which legitimate APIs do not provide for third-party tools), Approach A (IronGate) offers the best risk-adjusted option. Its Go binary architecture means the billing circumvention code is compiled and harder to detect than interpreted Python/Node.js. The OpenRelay-Go project has already merged openclaw-billing and hermes-billing, proving the technical viability. However, users should understand that this approach may break at any time when Anthropic updates its detection algorithms.

Approach B (SwarmRelay) is the most resilient architecture but also the most operationally complex. It is recommended only for teams that can dedicate DevOps resources to managing 6+ Docker containers and have the expertise to debug inter-service communication failures. Its key advantage is that when one provider's proxy breaks, the others continue serving, and the broken service can be hot-fixed without affecting the rest of the mesh.

Approach	Overall Score	Risk Level	Verdict
A: IronGate	7.5/10	High	Best for performance; accept legal risk
B: SwarmRelay	7.0/10	High	Best for resilience; accept complexity
C: LegitBridge	8.5/10	Low	Best for long-term; accept limited free access

Table 6: Final integration approach verdict with overall scoring.